

L'architecture Transformer de A à Z

Chapitre 1

Pourquoi comprendre l'architecture ?

Les choix que vous ferez en TP ne sont pas arbitraires

- Pourquoi $r=8$ pour LoRA et pas $r=64$?
- Pourquoi adapter Q et V mais pas K ?
- Pourquoi un encodeur seul pour la NER, mais encodeur-décodeur pour la normalisation ?

Ces questions trouvent toutes leur réponse dans le mécanisme d'attention — c'est l'objet de ce chapitre.

Plan du chapitre

- 1. Les limites du paradigme séquentiel (RNN, LSTM)
- 2. Le mécanisme d'attention (Q, K, V)
- 3. Multi-Head Attention
- 4. L'encodage positionnel (sinusoïdal, RoPE, ALiBi)
- 5. Architecture complète du Transformer
- 6. Trois familles : encoder-only, decoder-only, encoder-decoder
- 7. Tokenisation (BPE, SentencePiece) et le cas du moyen français
- 8. Cas pratique : visualiser les têtes d'attention
- 9. Connexion aux décisions du module

PARTIE 1

Les limites du paradigme séquentiel : RNN et LSTM

Avant les Transformers : les RNN

Un état caché qui résume tout le passé

$$h_t = f(W_h \cdot h_{t-1} + W_x \cdot x_t + b)$$

- Pour traiter une séquence $(x_1, \dots, x_T)$, on maintient un vecteur h_t qui agrège l'information passée
- Dimension fixe (souvent 512 ou 1024) : tout le contexte doit être compressé dans ce vecteur
- C'est une contrainte architecturale, pas un défaut d'implémentation

Le « vanishing gradient »

La dépendance longue distance devient inapprenable

- À la rétropropagation, le gradient se multiplie à chaque pas de temps
- Si chaque facteur < 1 (cas fréquent avec sigmoïde / tanh), le gradient décroît exponentiellement
- Exemple : facteur 0.9 sur 50 pas $\rightarrow 0.9^{50} \approx 0.005$

Conséquence : relier le sujet d'une phrase à son verbe, séparés par une longue proposition relative, devient pratiquement inaccessible à l'apprentissage.

Les LSTM (portes oubli / entrée / sortie) atténuent partiellement ce problème via une cellule mémoire séparée — mais ne le résolvent pas pour les très longues séquences.

La séquentialité : un goulot d'étranglement

Impossible de paralléliser le calcul sur GPU

- h_t dépend de h_{t-1} , qui dépend de h_{t-2} , etc.
- Entraînement : le temps de calcul croît linéairement avec la longueur de la séquence
- Inférence sur de longs textes : même contrainte

2017 — "**Attention Is All You Need**" (Vaswani et al.) : abandon de la récurrence au profit d'un mécanisme d'attention entièrement parallélisable, en une seule opération matricielle.

PARTIE 2

Le mécanisme d'attention : queries, keys, values

Intuition : à quoi se réfère « son » ?

"Le roi signa l'acte en son palais royal."

- Chaque token pose une question (query) aux autres tokens
- Chaque token expose une clé (key) : à quel point répond-il à cette question ?
- Chaque token porte une valeur (value) : le contenu qu'il partage si sa clé est pertinente

La sortie pour un token est une combinaison pondérée des valeurs de tous les autres tokens, où les poids dépendent de la compatibilité query / key.

Trois projections apprises : Q, K, V

Chaque token est projeté dans trois espaces distincts

$$Q = X W^Q \quad K = X W^K \quad V = X W^V$$

- X contient les vecteurs des n tokens de la séquence (dimension d_{model})
- W^Q et W^K projettent vers une dimension d_k ; W^V vers une dimension d_v
- Ces matrices de poids sont apprises pendant l'entraînement

Scaled Dot-Product Attention

La formule centrale du Transformer

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^t}{\sqrt{d_k}}\right) V$$

QK^t

Matrice $n \times n$: similarité entre la query du token i et la key du token j

$\div \sqrt{d_k}$

Facteur d'échelle : évite que la softmax sature quand d_k est grand (64-128)

$\times V$

Somme pondérée des valeurs : pour chaque token i , $\text{out}_i = \sum a_{ij} \cdot v_j$

Coût : complexité quadratique

Le prix de l'attention complète

Temps : $O(n^2 \cdot d_k)$ Mémoire : $O(n^2)$

- n = longueur de la séquence : doubler n quadruple le coût en mémoire de la matrice d'attention
- C'est la principale limitation des Transformers pour les très longs documents
- Les variantes à attention « efficiente » (sparse, linéaire) cherchent à réduire ce coût

Illustration : la matrice d'attention

"Le roi signa l'acte en son palais royal."

	Le	roi	signa	l'	acte	en	son	palais	royal
roi	0.0	0.8	0.0	0.0	0.1	0.0	0.0	0.0	0.1
signa	0.0	0.4	0.1	0.0	0.3	0.0	0.0	0.0	0.0
son	0.0	0.7	0.0	0.0	0.0	0.0	0.1	0.1	0.0

La ligne « son » montre que le modèle apprend à connecter le pronom possessif à « roi » — c'est la résolution de coréférence émergente par attention.

La ligne « signa » montre une attention répartie entre « roi » (sujet) et « acte » (objet) — structure argumentale du verbe.

Implémentation en PyTorch

scaled_dot_product_attention — un extrait minimal

```
def scaled_dot_product_attention(Q, K, V, mask=None):  
    d_k = Q.size(-1)  
    scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(d_k)  
  
    if mask is not None:  
        scores = scores.masked_fill(mask == 0, float('-inf'))  
  
    attn_weights = F.softmax(scores, dim=-1)  
    output = torch.matmul(attn_weights, V)  
    return output, attn_weights
```

PARTIE 3

Multi-Head Attention : pourquoi plusieurs têtes ?

Le problème d'une seule tête

Une phrase porte plusieurs types de relations à la fois

Syntaxiques

accord sujet-verbe

Sémantiques

coréférence (pronoms, entités)

Discursives

cohésion thématique du texte

Un seul ensemble de paramètres (W^Q , W^K , W^V) ne peut pas capturer simultanément ces relations hétérogènes.

Solution : exécuter plusieurs opérations d'attention en parallèle, chacune dans un sous-espace de dimension réduite — les « têtes ».

La formule Multi-Head

Concaténation de h têtes parallèles

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \cdot W^O$$

$$\text{head}_i = \text{Attention}(Q \cdot W_i^Q, K \cdot W_i^K, V \cdot W_i^V)$$

- Chaque tête opère dans un sous-espace de dimension réduite $d_k = d_v = d_{\text{model}} / h$
- Exemple — BERT-base : $d_{\text{model}} = 768$, $h = 12$, $d_k = d_v = 64 \rightarrow h \times d_v = 768$

Les têtes se spécialisent

Une spécialisation émergente, non imposée

- Certaines têtes capturent des relations syntaxiques (dépendance objet-verbe)
- D'autres résolvent les coréférences pronominales
- D'autres encore capturent la position relative (tokens adjacents)
- Des têtes profondes capturent des relations sémantiques abstraites

Cette spécialisation émerge de l'entraînement — elle n'est pas conçue à la main. C'est ce qui rend les Transformers à la fois puissants et difficiles à interpréter (Clark et al. 2019 ; Voita et al. 2019).

Lien direct avec LoRA

Pourquoi adapter Q et V , mais pas K ?

W^Q — la requête

Contrôle ce que chaque token cherche dans le contexte → adapter W^Q réoriente l'attention vers des patterns pertinents pour la tâche.

W^V — le contenu

Contrôle ce qui est récupéré et propagé aux couches suivantes → adapter W^V change l'information transmise.

W^K — la comparaison

Rôle plus passif. Les gains à l'adapter sont souvent marginaux par rapport au coût en paramètres.

Ne pas adapter W^K n'est pas arbitraire : c'est un choix architectural justifié par la compréhension des rôles respectifs des projections.

PARTIE 4

L'encodage positionnel : comment un modèle sans récurrence connaît l'ordre

Un problème fondamental

L'attention est invariante par permutation

- Si l'on mélange les tokens d'une séquence, la matrice QK^t change
- Mais le mécanisme d'attention lui-même ne « sait » pas dans quel ordre les tokens se trouvaient
- Il faut donc injecter explicitement l'information de position

Trois approches majeures : l'encodage sinusoïdal additif (original), RoPE (rotation), et ALiBi (biais sur les scores).

Encodage sinusoïdal (Vaswani et al., 2017)

Un vecteur de position ajouté à chaque embedding

$$PE(t, 2i) = \sin(t / 10000^{(2i/d_model)})$$

$$PE(t, 2i+1) = \cos(t / 10000^{(2i/d_model)})$$

- t = position du token, i = indice de dimension
- Hautes fréquences $\rightarrow$ position fine ; basses fréquences $\rightarrow$ position globale

Propriété clé : pour tout décalage δ fixé, $PE(t+\delta)$ est une transformation linéaire de $PE(t)$ — le modèle peut raisonner sur les distances relatives.

RoPE — Rotary Position Embedding

Su et al., 2021 — utilisé par LLaMA, Mistral, etc.

- Plutôt que d'ajouter un vecteur de position, RoPE fait pivoter les vecteurs query et key d'un angle proportionnel à leur position
- Le produit scalaire $q_t' \cdot k_s'$ dépend alors uniquement de la différence de position $t - s$

$$q_t' = R_t q_t \quad k_s' = R_s k_s \quad \rightarrow \quad q_t' \cdot k_s' = q_t^t R_{-(t-s)} k_s$$

Avantages : extrapolation à des séquences plus longues qu'à l'entraînement, et préservation naturelle de la distance relative.

ALiBi — Attention with Linear Biases

Press et al., 2022

$$\text{score}_{ij} = (q_i \cdot k_j) / \sqrt{d_k} - m \cdot |i - j|$$

- Un biais négatif linéaire est ajouté aux scores, proportionnel à la distance entre tokens
- m est une pente spécifique à chaque tête d'attention
- Les tokens éloignés sont naturellement défavorisés → meilleure généralisation aux séquences longues

Pour votre corpus : RoPE et ALiBi gèrent mieux l'extrapolation de longueur — pertinent pour des textes médiévaux (chartes, registres) de longueurs très variables.

PARTIE 5

Architecture complète du Transformer

Le bloc Transformer

N blocs identiques empilés

1 — Multi-Head Self-Attention

+ connexion résiduelle + Layer Norm

2 — Feed-Forward Network (FFN)

+ connexion résiduelle + Layer Norm

$$\text{out}_1 = \text{LayerNorm}(x + \text{MultiHead}(x, x, x))$$

$$\text{out}_2 = \text{LayerNorm}(\text{out}_1 + \text{FFN}(\text{out}_1)) \quad \text{où } \text{FFN}(x) = W_2 \cdot \text{GELU}(W_1 x + b_1) + b_2$$

Connexions résiduelles et taille du FFN

- La connexion résiduelle (He et al., 2016) crée un chemin de gradient direct entre les couches
- Elle évite le vanishing gradient — le problème même que les RNN ne savaient pas résoudre
- La Layer Norm stabilise les activations
- d_{ff} (dimension du FFN) est généralement 4× plus grand que d_{model} — 3072 pour BERT-base ($d_{model} = 768$)

Vue d'ensemble : encodeur et décodeur

Empilement de N blocs, chacun avec attention puis FFN

ENCODER (×N)

Input Embeddings + Positional Encoding

Multi-Head Self-Attention (+ Add & Norm)

Feed-Forward Network (+ Add & Norm)

DÉCODEUR (×N)

Output Embeddings (décalés) + Positional Encoding

Masked Multi-Head Self-Attention (+ Add & Norm)

Cross-Attention : $Q \leftarrow$ décodeur, $K/V \leftarrow$ encodeur
(+ Add & Norm)

Feed-Forward Network (+ Add & Norm)

La cross-attention permet au décodeur, à chaque étape de génération, de « consulter » n'importe quelle partie de la séquence d'entrée.

PARTIE 6

Trois familles architecturales et leurs usages

Une taxonomie opérationnelle

Pas abstraite : c'est la raison de vos choix de modèles dans ce module

- CamemBERT pour la NER médiévale (Jour 3)
- T5 / mT5 pour la normalisation orthographique (Jour 2)
- GPT / LLaMA pour le prompting (Jour 4)

Chaque famille correspond à un mode d'attention différent — bidirectionnelle, causale, ou les deux + cross-attention.

Encoder-only : BERT et descendants

Attention bidirectionnelle — chaque token voit tous les autres

- Pré-entraînement : Masked Language Modeling (15% des tokens masqués) + Next Sentence Prediction
- Usages : classification de tokens (NER, POS), classification de séquences, extraction de features
- Modèles : BERT, RoBERTa, DeBERTa, CamemBERT, XLM-RoBERTa

CamemBERT (138 Go de français, OSCAR) sera utilisé pour la NER au Jour 3 : pour décider si « Guillaume » est PER ou TITLE, le contexte gauche ET droit est nécessaire.

Decoder-only : GPT et les modèles génératifs

Attention causale — un token ne voit que ses prédécesseurs

$$M_{ij} = 0 \text{ si } j \leq i ; -\infty \text{ si } j > i$$

- Le masque causal garantit que la softmax attribue un poids nul aux tokens futurs
- Pré-entraînement : Next Token Prediction — objectif simple, dataset massif
- Usages : génération de texte, complétion, dialogue, few-shot prompting
- Modèles : GPT-2/3/4, LLaMA, Mistral, Falcon

Encoder-decoder : T5, BART, mT5

Bidirectionnel en entrée, causal + cross-attention en sortie

`CrossAttention( Q_dec , K_enc , V_enc )`

- L'encodeur lit l'entrée en bidirectionnel ; le décodeur génère token par token via self-attention causale + cross-attention
- Usages idéaux : traduction, résumé, normalisation orthographique, correction, QA génératif
- Modèles : T5, mT5, BART, mBART, PEGASUS

mT5 sera votre modèle de base pour la normalisation du moyen français (Jour 2) : token brut → forme normalisée, une transduction de séquence.

Tableau de synthèse

Famille	Attention	Tâches typiques	Modèles	Usage dans ce module
Encoder-only	Bidirectionnelle	NER, classification, extraction	BERT, CamemBERT	NER médiévale (Jour 3)
Decoder-only	Causale	Génération, complétion	GPT, LLaMA	Prompting (Jour 4)
Encoder-decoder	Bidir. + Cross	Traduction, résumé, normalisation	T5, mT5, BART	Normalisation (Jour 2)

PARTIE 7

Tokenisation : BPE, SentencePiece et les enjeux pour le moyen français

Pourquoi tokeniser ?

Le pont entre texte brut et vocabulaire fini

- Les Transformers opèrent sur des tokens discrets, pas sur des caractères bruts
- La qualité de représentation dépend de la qualité du token : un mot entier ou un sous-mot fréquent est mieux représenté qu'un fragment isolé

Conséquence directe : un mot fragmenté en sous-sous-mots inconnus perd l'accès aux représentations pré-entraînées riches.

Byte Pair Encoding (BPE)

Sennrich et al., 2016 — l'algorithme le plus répandu

- 1. Initialiser le vocabulaire avec tous les caractères du corpus
- 2. Compter les paires de symboles adjacents les plus fréquentes
- 3. Fusionner la paire la plus fréquente en un nouveau symbole
- 4. Répéter jusqu'à atteindre la taille de vocabulaire cible

Corpus initial : "low lower lowest new newer newest"

Fusion 1 : (e, r) -> er [paire la plus fréquente]

Fusion 2 : (lo, w) -> low

...

Résultat : "low", "er", "est", "new" deviennent des tokens entiers

SentencePiece et le modèle Unigram

Kudo & Richardson, 2018 — utilisé par CamemBERT et T5

- SentencePiece est un framework qui implémente BPE, mais aussi le modèle Unigram
- Unigram part d'un grand vocabulaire et supprime itérativement les tokens qui minimisent la perte de log-vraisemblance
- Différence clé : l'espace est traité comme un caractère ordinaire () → tokenisation indépendante des espaces

Utile pour les langues sans espaces, ou pour des textes à espacement irrégulier — comme les manuscrits médiévaux.

Le défi du moyen français

Des tokeniseurs entraînés sur le français moderne

Graphies non standardisées

« roi » s'écrit aussi roy, roÿ, roys, rei...

Abréviations manuscrites

tilde de nasalité (q~ = que),
signes suprascripts

Agglutination / contractions

dou (= du), au (= à le), ès (= en les)

Ces variantes graphiques et abréviations sont systématiquement absentes du vocabulaire BPE du français moderne (CamemBERT, RoBERTa).

Conséquence mesurable : le taux OOV (Out-of-Vocabulary) sur le moyen français est 2 à 5 fois supérieur à celui du français moderne — 15 à 30% sur CREMMA Medieval selon les genres.

Exemple : fragmentation avec CamemBERT

Même phrase, français moderne vs moyen français

```
tokenizer = AutoTokenizer.from_pretrained("almanach/camembert-base")
```

```
# Français moderne
```

```
tokenizer.tokenize("Le roi signa l'acte dans son palais.")
```

```
# ['_Le', '_roi', '_signa', '_l', '', 'acte', '_dans', '_son', '_palais', '.']
```

```
# Moyen français
```

```
tokenizer.tokenize("Li roys signa l'acte en son palloys roial.")
```

```
# ['_Li', '_ro', 'ys', '_signa', '_l', '', 'acte', '_en', '_son', '_pa', 'lo', 'ys', '_ro', 'ial', '.']
```

```
# -> "roys" en 2 tokens, "palloys" en 3, "roial" en 2
```

Conséquences et point de vigilance

Avant de choisir votre modèle de base

- Un token fragmenté (`_ro`, `ys`) ne bénéficie pas de l'embedding pré-entraîné du mot entier
- Le modèle doit « réapprendre » à composer le sens de ces sous-mots
- En NER, une entité fragmentée en 4-5 sous-tokens est plus difficile à détecter

Avant de choisir votre modèle de base, quantifiez le taux OOV sur votre corpus spécifique — ce sera l'une des premières analyses de l'EDA du Jour 1. Ne choisissez pas CamemBERT « par défaut ».

PARTIE 8

Cas pratique : visualiser les têtes d'attention sur un texte médiéval

Pourquoi visualiser l'attention ?

Une fenêtre partielle — et parfois trompeuse

- Vérifier que le modèle capte des relations linguistiques sensées
- Identifier des pathologies (têtes « entropiques » qui distribuent l'attention uniformément)
- Motiver les choix d'interprétabilité dans un rapport destiné à des humanistes

Mise en garde fondamentale (Jain & Wallace, 2019) : les poids d'attention ne sont pas des explications causales. D'autres mécanismes — connexions résiduelles, FFN — contribuent aussi au résultat.

Pipeline de visualisation

Extraire et afficher une tête d'attention donnée

```
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModel.from_pretrained(model_name, output_attentions=True)

inputs = tokenizer(text, return_tensors="pt")
outputs = model(**inputs)

# outputs.attentions : tuple (batch, n_heads, seq, seq) par couche
attn = outputs.attentions[layer][0, head].numpy()

# Affichage avec matplotlib (imshow, cmap="Blues")
visualize_attention("Li roys de France... firent accord.", layer=11, head=3)
```

Que faut-il observer ?

Les rôles des têtes selon la profondeur de la couche

Couches 1-4 (premières)

Attention aux tokens adjacents et aux ponctuations — rôle syntaxique local

Couches 5-8 (intermédiaires)

Attention plus diffuse, relations à moyenne portée

Couches 9-12 (dernières)

Relations sémantiques abstraites, coréférence, structure argumentale

Pour un texte médiéval, comparez avec un texte français moderne équivalent : les différences de patterns d'attention reflètent la désorientation du modèle face à une graphie non vue à l'entraînement.

PARTIE 9

Connexion aux décisions du module

Récapitulatif : architecture → choix pratiques

Question du module	Réponse architecturale
Pourquoi CamemBERT pour NER ?	Attention bidirectionnelle nécessaire pour la classification de tokens
Pourquoi mT5 pour normalisation ?	Cross-attention encoder-decoder pour la transduction de séquences
Pourquoi LoRA sur Q et V ?	Q contrôle ce que l'on cherche, V contrôle ce que l'on récupère
Pourquoi r=8 souvent suffisant ?	Le rang intrinsèque des mises à jour de fine-tuning est faible
Pourquoi quantifier l'OOV d'abord ?	Un taux OOV élevé dégrade la qualité des représentations pré-entraînées
Pourquoi RoPE pour l'extrapolation ?	Distance relative encodée dans l'angle de rotation, pas la position absolue

Ce que vous verrez au Jour 2

LoRA et l'hypothèse de faible rang

- Vous choisirez $r=8$ ou $r=16$ pour votre expérience d'ablation LoRA
- r contrôle la dimension du sous-espace de mise à jour que vous autorisez
- Un r faible suppose que les ajustements nécessaires (normalisation du moyen français) vivent dans un espace de faible dimension

Cette hypothèse est souvent vérifiée empiriquement — et vous la testerez directement avec vos données.

Ce qu'il faut retenir

- L'attention remplace la récurrence : calcul parallélisable, mais coût quadratique en longueur
- Q, K, V et le scaling par $\sqrt{d_k}$ structurent le mécanisme de base
- Multi-Head : spécialisation émergente — base du raisonnement LoRA (Q/V vs K)
- L'encodage positionnel (sinusoïdal, RoPE, ALiBi) injecte l'ordre, absent de l'attention pure
- Encoder-only / decoder-only / encoder-decoder : trois familles, trois usages distincts
- La tokenisation conditionne la qualité des représentations — vigilance particulière pour le moyen français